

```
# Install necessary libraries
!pip install chromadb
!pip install sentence-transformers
```

Requirement already satisfied: python-dateutil>=2.5.3 in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.1.0->chromadb)

Requirement already satisfied: websocket-client!=0.40.0,!0.41.*,!=0.42.*,>=0.32.0 in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.1.0->chromadb)

Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.1.0->chromadb) (2.32.4)

Requirement already satisfied: requests-oauthlib in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.1.0->chromadb)

Requirement already satisfied: urllib3!=2.6.0,>=1.24.2 in /usr/local/lib/python3.12/dist-packages (from kubernetes>=28.1.0->chromadb)

Collecting durationpy>=0.7 (from kubernetes>=28.1.0->chromadb)

Downloading durationpy-0.10-py3-none-any.whl.metadata (340 bytes)

Requirement already satisfied: flatbuffers in /usr/local/lib/python3.12/dist-packages (from onnxruntime>=1.14.1->chromadb) (25.2.9)

Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from onnxruntime>=1.14.1->chromadb) (5.29.3)

Requirement already satisfied: sympy in /usr/local/lib/python3.12/dist-packages (from onnxruntime>=1.14.1->chromadb) (1.14.0)

Requirement already satisfied: importlib-metadata<8.8.0,>=6.0 in /usr/local/lib/python3.12/dist-packages (from opentelemetry-api>=1.24.0->chromadb)

Requirement already satisfied: googleapis-common-protos<=1.57 in /usr/local/lib/python3.12/dist-packages (from opentelemetry-exporter-otlp-proto-grpc>=1.2.0->chromadb)

Collecting opentelemetry-exporter-otlp-proto-common==1.40.0 (from opentelemetry-exporter-otlp-proto-grpc==1.2.0->chromadb)

Downloading opentelemetry_exporter_otlp_proto_common-1.40.0-py3-none-any.whl.metadata (1.9 kB)

Collecting opentelemetry-proto==1.40.0 (from opentelemetry-exporter-otlp-proto-grpc==1.2.0->chromadb)

Downloading opentelemetry_proto-1.40.0-py3-none-any.whl.metadata (2.4 kB)

Collecting opentelemetry-sdk>=1.2.0 (from chromadb)

Downloading opentelemetry_sdk-1.40.0-py3-none-any.whl.metadata (1.6 kB)

Collecting opentelemetry-api>=1.2.0 (from chromadb)

Downloading opentelemetry_api-1.40.0-py3-none-any.whl.metadata (1.5 kB)

Collecting opentelemetry-semantic-conventions==0.61b0 (from opentelemetry-sdk>=1.2.0->chromadb)

Downloading opentelemetry_semantic_conventions-0.61b0-py3-none-any.whl.metadata (2.5 kB)

Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic>=2.0->chromadb)

Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic>=2.0->chromadb)

Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic>=2.0->chromadb)

Requirement already satisfied: python-dotenv>=0.21.0 in /usr/local/lib/python3.12/dist-packages (from pydantic-settings>=2.0->chromadb)

Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=10.11.0->chromadb)

Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=10.11.0->chromadb)

Requirement already satisfied: huggingface-hub<2.0,>=0.16.4 in /usr/local/lib/python3.12/dist-packages (from tokenizers>=0.13.0->chromadb)

Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer>=0.9.0->chromadb) (8.3.1)

Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer>=0.9.0->chromadb) (1.5.4)

Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer>=0.9.0->chromadb) (0.0.2)

Requirement already satisfied: httpxtools>=0.6.3 in /usr/local/lib/python3.12/dist-packages (from uvicorn[standard]>=0.18.3->chromadb)

Requirement already satisfied: uvloop>=0.15.1 in /usr/local/lib/python3.12/dist-packages (from uvicorn[standard]>=0.18.3->chromadb)

Requirement already satisfied: watchfiles>=0.20 in /usr/local/lib/python3.12/dist-packages (from uvicorn[standard]>=0.18.3->chromadb)

Requirement already satisfied: websockets>=10.4 in /usr/local/lib/python3.12/dist-packages (from uvicorn[standard]>=0.18.3->chromadb)

Requirement already satisfied: filelock>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.16.4->chromadb)

Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.16.4->chromadb)

Requirement already satisfied: hf-xet<2.0.0,>=1.4.2 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.16.4->chromadb)

Requirement already satisfied: zipp>=3.20 in /usr/local/lib/python3.12/dist-packages (from importlib-metadata<8.8.0,>=6.0->opentelemetry-api>=1.24.0->chromadb)

Requirement already satisfied: mdurl~0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=10.11.0->chromadb)

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests-oauthlib>=1.3.1->chromadb)

Requirement already satisfied: oauthlib>=3.0.0 in /usr/local/lib/python3.12/dist-packages (from requests-oauthlib>=1.3.1->chromadb)

Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy->onnxruntime>=1.14.1->chromadb)

Downloading chromadb-1.5.6-cp39-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (22.4 MB)

22.4/22.4 MB 50.5 MB/s eta 0:00:00

Downloading bcrypt-5.0.0-cp39-abi3-manylinux_2_34_x86_64.whl (278 kB)

278.2/278.2 kB 18.1 MB/s eta 0:00:00

Downloading build-1.4.2-py3-none-any.whl (24 kB)

Downloading kubernetes-35.0.0-py2.py3-none-any.whl (2.0 MB)

2.0/2.0 MB 32.2 MB/s eta 0:00:00

Downloading onnxruntime-1.24.4-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (17.2 MB)

17.2/17.2 MB 47.6 MB/s eta 0:00:00

Downloading opentelemetry_exporter_otlp_proto_grpc-1.40.0-py3-none-any.whl (20 kB)

Downloading opentelemetry_exporter_otlp_proto_common-1.40.0-py3-none-any.whl (18 kB)

Downloading opentelemetry_proto-1.40.0-py3-none-any.whl (72 kB)

72.1/72.1 kB 5.3 MB/s eta 0:00:00

Downloading opentelemetry_sdk-1.40.0-py3-none-any.whl (141 kB)

```
# Import libraries
import chromadb
from sentence_transformers import SentenceTransformer
```

```
client = chromadb.Client()
collection = client.create_collection(name="rag_collection", get_or_create=True)
print(f"ChromaDB collection '{collection.name}' initialized.")
```

```
ChromaDB collection 'rag_collection' initialized.
```

```
model = SentenceTransformer('all-MiniLM-L6-v2')
print("SentenceTransformer model 'all-MiniLM-L6-v2' loaded.")
```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
modules.json: 100% 349/349 [00:00<00:00, 36.6kB/s]

config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 12.1kB/s]

README.md: 10.5k/? [00:00<00:00, 889kB/s]

sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 870B/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster d
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to en
config.json: 100% 612/612 [00:00<00:00, 15.1kB/s]

model.safetensors: 100% 90.9M/90.9M [00:01<00:00, 209MB/s]

Loading weights: 100% 103/103 [00:00<00:00, 379.47it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key | Status | |
-----+-----+---
embeddings.position_ids | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
tokenizer_config.json: 100% 350/350 [00:00<00:00, 10.9kB/s]

vocab.txt: 232k/? [00:00<00:00, 5.16MB/s]

tokenizer.json: 466k/? [00:00<00:00, 15.9MB/s]

special_tokens_map.json: 100% 112/112 [00:00<00:00, 1.56kB/s]

config.json: 100% 190/190 [00:00<00:00, 4.59kB/s]
SentenceTransformer model 'all-MiniLM-L6-v2' loaded.

```

```

documents = [
    "Artificial Intelligence is the simulation of human intelligence.",
    "Machine Learning is a subset of AI.",
    "Deep Learning uses neural networks.",
    "Natural Language Processing deals with text data.",
    "RAG combines retrieval and generation."
]

ids = ["doc1", "doc2", "doc3", "doc4", "doc5"]

print("Sample documents prepared.")

```

Sample documents prepared.

```

embeddings = model.encode(documents)
print(f"Converted {len(documents)} documents into {embeddings.shape[1]}-dimensional embeddings.")

```

Converted 5 documents into 384-dimensional embeddings.

```

collection.add(
    documents=documents,
    embeddings=embeddings.tolist(), # Convert numpy array to list for ChromaDB
    ids=ids
)
print(f"Added {len(documents)} documents and their embeddings to ChromaDB collection '{collection.name}'.")

```

Added 5 documents and their embeddings to ChromaDB collection 'rag_collection'.

```

query = "What is AI?"

# Encode the query into an embedding
query_embedding = model.encode([query])

# Perform similarity search in ChromaDB
results = collection.query(
    query_embeddings=query_embedding.tolist(), # Convert numpy array to list
    n_results=2 # Retrieve the top 2 most similar documents
)

```

```
print("Similarity search results:")
print(results)
```

```
Similarity search results:
{'ids': [['doc1', 'doc2']], 'embeddings': None, 'documents': [['Artificial Intelligence is the simulation of human intelligence.
```

```
print("Retrieved Documents:")
for doc in results['documents'][0]:
    print(f"- {doc}")
```

```
Retrieved Documents:
- Artificial Intelligence is the simulation of human intelligence.
- Machine Learning is a subset of AI.
```

```
def retrieve(query):
    query_embedding = model.encode([query])
    results = collection.query(
        query_embeddings=query_embedding.tolist(),
        n_results=2 # You can adjust this number
    )
    return results['documents'][0]

print("Testing retrieval function for 'Explain machine learning':")
retrieved_ml_docs = retrieve("Explain machine learning")
for doc in retrieved_ml_docs:
    print(f"- {doc}")
```

```
Testing retrieval function for 'Explain machine learning':
- Machine Learning is a subset of AI.
- Deep Learning uses neural networks.
```